

实验四 矩阵键盘扫描控制电路设计报告

姓名：怀迪 学号：PB23061112

一、实验内容简述

本次实验要求我们编写 Verilog 代码来实现数码管根据所按的按键显示对应的数字或者字母的功能，并在 Quartus 软件中进行仿真和 FPGA 验证。

二、设计分析

1. 实验题目分析

我们需要以行信号为输入，列信号为输出，设计一个矩阵键盘扫描控制电路。矩阵键盘的工作原理是通过行列扫描来检测按键的状态。具体来说，当按下某个按键时，对应的行和列信号会被连接，从而形成一个闭合电路。我们需要设计一个电路来扫描这些行列信号，并根据按下的按键输出对应的数字或字母。

①在时钟上升沿循环输出列信号；

②同时在时钟下降沿检测行信号，当行信号有一行输出为 0 时，说明此时列信号循环到了相应的输出；

③此时就可以根据行列信号的组合来确定按下的按键，并输出对应的数字或字母。

2. 编程思路

我把代码分成了五个模块：实现具体功能的顶层模块，分频模块，列信号产生模块，行信号检测模块，数码管解码模块。

2.1 顶层模块

顶层模块只需调用其他模块即可实现具体功能的实现。

首先调用分频模块对 clk 信号进行分频,得到一个较低频率的时钟信号 clk_divided,本次实验我选择将时钟分频成 1000Hz。

然后调用列信号产生模块和行信号检测模块。

最后调用数码管解码模块显示所按按键。

2.2 分频模块

分频模块主要实现对输入的 clk 信号进行分频操作。

首先输入未分频的时钟信号,使用一个计数器对输入的 clk 信号进行计数,当计数器达到设定的分频值时,翻转输出的分频信号,并将计数器清零。

此时得到的输出信号为分频后的时钟信号 clk_divided。

2.3 列信号产生模块

在每个时钟上升沿进行 0-3 的循环计数,根据计数值输出对应的列信号。当计数值为 0 时,输出列信号为 4'b1110;当计数值为 1 时,输出列信号为 4'b1101;当计数值为 2 时,输出列信号为 4'b1011;当计数值为 3 时,输出列信号为 4'b0111。

2.4 行信号检测模块

以列信号输出的信号和按键信号作为输入,在每个时钟的下降沿检测行信号。当行信号有一行输出为 0 时,说明此时列信号循环到了相应的输出,此时根据行列信号的组合来确定按下的按键,并输出对应的数字或字母。

2.5 数码管解码模块

根据行列信号的组合来确定按下的按键,并输出对应的数字或字母。

三、VHDL 源代码

3.1 顶层模块

```

1 | `timescale 10ns/1ns
2 |
3 | module FPGA_EXP4_hd (
4 |     input clk,
5 |     input [3:0] row_signal, //行信号
6 |     output [3:0] col_signal, //列信号
7 |     output [6:0] LED_data, //数码管显示
8 |     output LED_choose
9 | );
10 | //分频
11 | wire clk_divided;
12 | FPGA_EXP4_div div(
13 |     .clk(clk),
14 |     .clkout(clk_divided)
15 | );
16 |
17 | //循环产生列控制信号
18 | col_generate col_generate(
19 |     .clk(clk_divided),
20 |     .col_signal(col_signal)
21 | );
22 |
23 | //读取行信号判断哪个按键被按下,并显示在数码管上
24 | row_scan row_scan(
25 |     .clk(clk_divided),
26 |     .col_signal(col_signal),
27 |     .row_signal(row_signal),
28 |     .LED_data(LED_data),
29 |     .LED_choose(LED_choose)
30 | );
31 |
32 |
33 | endmodule

```

3.2 分频模块

```

1 | `timescale 10ns/1ns
2 |
3 | module FPGA_EXP4_div(
4 |     input clk,
5 |     output reg clkout
6 | );
7 | // 分频 (通过计数器实现 50MHz~2Hz)
8 | reg [14:0] cnt; // 计数器
9 | always @(posedge clk)
10 |     begin
11 |         if (cnt >= 24999)
12 |             begin // 50MHz/2/25000 = 1000Hz
13 |                 cnt <= 1'b0;
14 |                 clkout <= ~clkout; // 翻转输出, 生成 1000Hz 方波
15 |             end
16 |         else
17 |             begin
18 |                 cnt <= cnt + 1'b1;
19 |             end
20 |         end
21 | endmodule

```

3.3 列信号产生模块

```

1 `timescale 10ns/1ns
2
3 module col_generate (
4     input clk,
5     output reg[3:0] col_signal//列信号
6 );
7     reg [1:0] num;//用来循环计数产生列控制信号
8     always @(posedge clk)//循环产生序列
9     begin
10         if(num == 2'b11)
11         begin
12             num <= 2'b00;
13         end
14         else if(num==2'b00 && num<=2'b10)
15         begin
16             num <= num + 2'b01;
17         end
18         else
19         begin
20             num <=2'b00;
21         end
22
23         case (num)
24             2'b00: begin col_signal <= 4'b1110; end
25             2'b01: begin col_signal <= 4'b1101; end
26             2'b10: begin col_signal <= 4'b1011; end
27             2'b11: begin col_signal <= 4'b0111; end
28             default: begin num <= 2'b11; end
29         endcase
30     end
31 endmodule

```

3.4 行信号检测模块

```

1 `timescale 10ns/1ns
2
3 module row_scan(
4     input clk,
5     input [3:0] col_signal,//列信号
6     input [3:0] row_signal,//行信号
7     output [6:0] LED_data,
8     output LED_choose
9 );
10     reg[3:0] position;
11     assign LED_choose=1'b1;
12     always @(negedge clk)//循环产生序列
13     begin
14         case (row_signal)
15             4'b1110:
16             begin
17                 position[3:2]<=2'b00;
18                 case (col_signal)
19                     4'b1110: position[1:0]<=2'b00;
20                     4'b1101: position[1:0]<=2'b01;
21                     4'b1011: position[1:0]<=2'b10;
22                     4'b0111: position[1:0]<=2'b11;
23                     4'b1111: position[1:0]<=position[1:0];
24                     default: position[1:0]<=2'b00;
25                 endcase
26             end
27             4'b1101:
28             begin
29                 position[3:2]<=2'b01;
30                 case (col_signal)
31                     4'b1110: position[1:0]<=2'b00;
32                     4'b1101: position[1:0]<=2'b01;
33                     4'b1011: position[1:0]<=2'b10;
34                     4'b0111: position[1:0]<=2'b11;
35                     4'b1111: position[1:0]<=position[1:0];
36                     default: position[1:0]<=2'b00;
37                 endcase
38             end

```

```

39 |         4'b1011:
40 |         begin
41 |             position[3:2]<=2'b10;
42 |             case (col_signal)
43 |                 4'b1110: position[1:0]<=2'b00;
44 |                 4'b1101: position[1:0]<=2'b01;
45 |                 4'b1011: position[1:0]<=2'b10;
46 |                 4'b0111: position[1:0]<=2'b11;
47 |                 4'b1111: position[1:0]<=position[1:0];
48 |                 default: position[1:0]<=2'b00;
49 |             endcase
50 |         end
51 |         4'b0111:
52 |         begin
53 |             position[3:2]<=2'b11;
54 |             case (col_signal)
55 |                 4'b1110: position[1:0]<=2'b00;
56 |                 4'b1101: position[1:0]<=2'b01;
57 |                 4'b1011: position[1:0]<=2'b10;
58 |                 4'b0111: position[1:0]<=2'b11;
59 |                 4'b1111: position[1:0]<=position[1:0];
60 |                 default: position[1:0]<=2'b00;
61 |             endcase
62 |         end
63 |         4'b1111: position<=position;
64 |         default: position<=4'b1101;
65 |     endcase
66 | end
67 |
68 |
69 | LED_decoder LED_decoder(
70 |     .in(position),
71 |     .out(LED_data)
72 | );
73 |
74 | endmodule

```

3.5 数码管解码模块

```

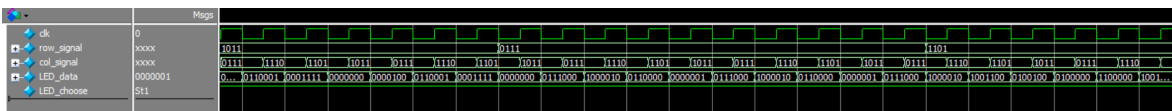
1 | `timescale 10ns/1ns
2 |
3 | module LED_decoder(
4 |     input [3:0] in,
5 |     output reg [6:0] out
6 | );
7 |     always @(*)
8 |     begin
9 |         case (in)
10 |             4'b0000: out = 7'b1001111; //1
11 |             4'b0001: out = 7'b0010010; //2
12 |             4'b0010: out = 7'b0000110; //3
13 |             4'b0011: out = 7'b0001000; //A
14 |             4'b0100: out = 7'b1001100; //4
15 |             4'b0101: out = 7'b0100100; //5
16 |             4'b0110: out = 7'b0100000; //6
17 |             4'b0111: out = 7'b1100000; //B
18 |             4'b1000: out = 7'b0001111; //7
19 |             4'b1001: out = 7'b0000000; //8
20 |             4'b1010: out = 7'b0000100; //9
21 |             4'b1011: out = 7'b0110001; //C
22 |             4'b1100: out = 7'b0110000; //E(*)
23 |             4'b1101: out = 7'b0000001; //0
24 |             4'b1110: out = 7'b0111000; //F(#)
25 |             4'b1111: out = 7'b1000010; //D
26 |         endcase
27 |     end
28 | endmodule

```

四、仿真结果记录

```
1 |`timescale 10ns/1ns
2 |
3 |module FPGA_EXP4_hd_tb ();
4 |    reg clk;
5 |    reg [3:0] row_signal;
6 |    wire[3:0] col_signal;
7 |    wire[6:0] LED_data;
8 |    wire LED_choose;
9 |
10 |    initial clk=0;
11 |    always #1 clk = ~clk; //产生时钟信号
12 |
13 |    initial begin
14 |        #20 row_signal<=4'b1110;
15 |        #20 row_signal<=4'b1101;
16 |        #20 row_signal<=4'b1011;
17 |        #20 row_signal<=4'b0111;
18 |        #20 row_signal<=4'b1101;
19 |    end
20 |
21 |    col_generate col_generate(
22 |        clk,
23 |        col_signal
24 |    );
25 |
26 |    row_scan row_scan(
27 |        .clk(clk),
28 |        .col_signal(col_signal),
29 |        .row_signal(row_signal),
30 |        .LED_data(LED_data),
31 |        .LED_choose(LED_choose)
32 |    );
33 |
34 |endmodule
```

仿真代码如上图所示。



上图部分仿真波形图,可以看到当 row_signal 和 col_signal 为不同值时,LED_data 会输出对应的数码管显示值,与预期相符。

五、FPGA 验证结果记录

我在实验箱上从上到下依次按下了 4 行 4 列矩阵键盘。当我按下矩阵键盘上的某个按键时,数码管上会显示对应的数字或字母,与理论相符。

六、实验总结

本次实验过程中比较顺利，并未遇到什么问题，唯一遇到比较棘手的问题是刚开始仿真波形时，输出一直不正确，后面我仔细检查了代码，可能是因为我时钟信号用顶层模块再次分频，导致时序出现问题，后面我把各个模块单独拿来使用，输出波形就正常了。